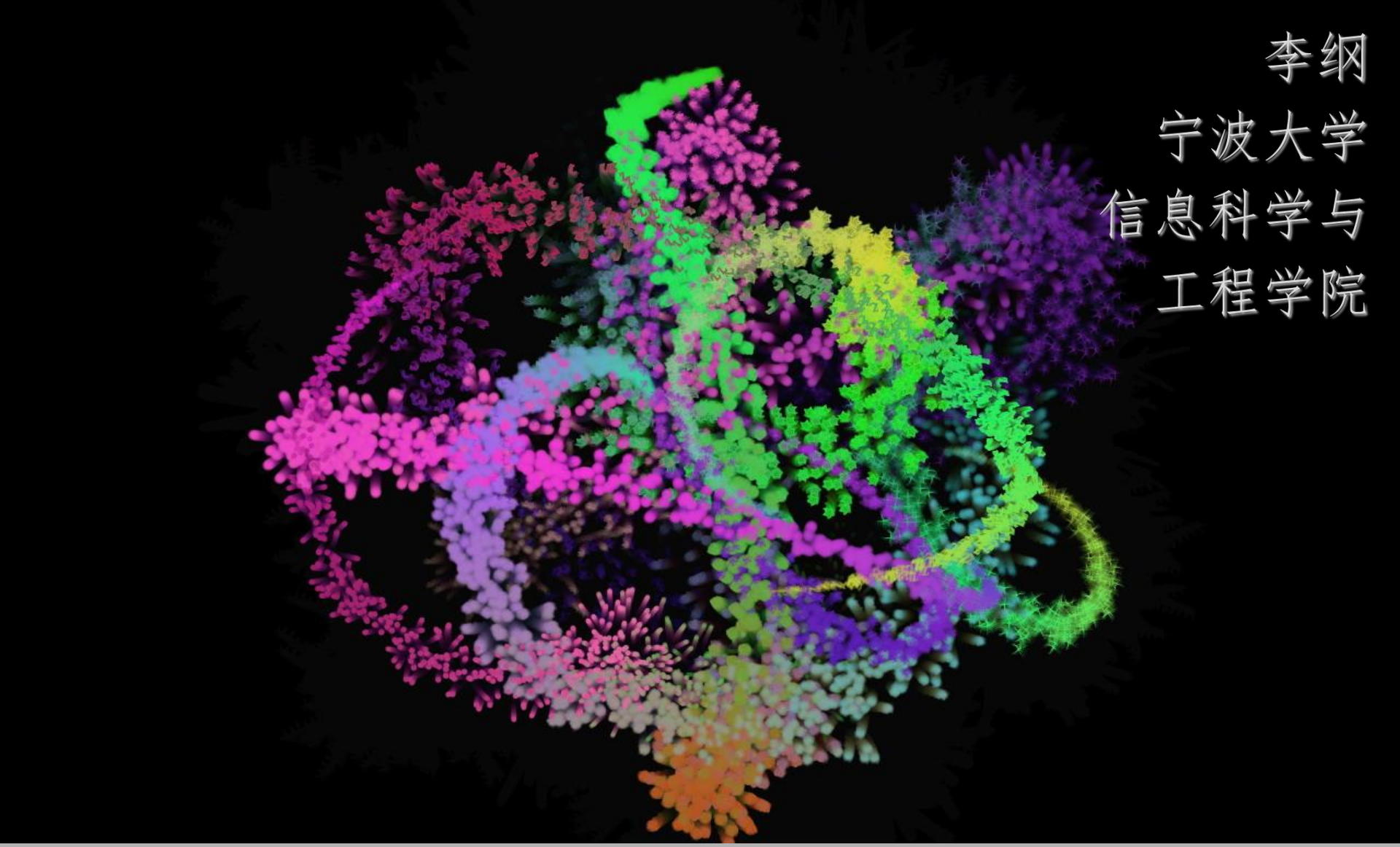


李纲
宁波大学
信息科学与
工程学院

概率算法 算法分析与设计

Analysis and Design of Computer Algorithms



- **学习要点**
- 理解产生伪随机数的算法
- 掌握数值概率算法的设计思想
- 掌握蒙特卡罗算法的设计思想
- 掌握拉斯维加斯算法的设计思想
- 掌握舍伍德算法的设计思想

随机化算法



4类：
数值随机化算法、蒙特卡罗算法、拉斯维加斯算法、舍伍德算法

数值随机化算法：近似解
蒙特卡罗算法：解未必正确、逼近
拉斯维加斯算法：正确解、会找不到解（可能效率低，足够时间会找到；改进有效性）
一旦找到一个解，那么这个解一定是问题的正确解。
舍伍德算法：正确解、总能找到，时间复杂性均衡，消除特例对算法效率的影响

随机数



随机数在概率算法设计中扮演着十分重要的角色。在现实计算机上无法产生真正的随机数，因此在概率算法中使用的随机数都是一定程度上随机的，即伪随机数。

线性同余法是产生伪随机数的最常用的方法。由线性同余法产生的随机序列$a_0,a_1,\ldots,a_n$满足

$$\begin{cases} a_0 = d \\ a_n = (ba_{n-1} + c) \bmod m \qquad n = 1,2,\cdots \end{cases}$$

TC
产生的随机数差不多？

其中b≥0，c≥0，d≤m。d称为该随机序列的种子。如何选取该方法中的常数b、c和m直接关系到所产生的随机序列的随机性能。这是随机性理论研究的内容，已超出讨论的范围。从直观上看，m应取得充分大，因此可取m为机器大数，另外应取gcd(m,b)=1，因此可取b为一素数。

随机数



P237：随机数
P238：硬币

好的随机数?

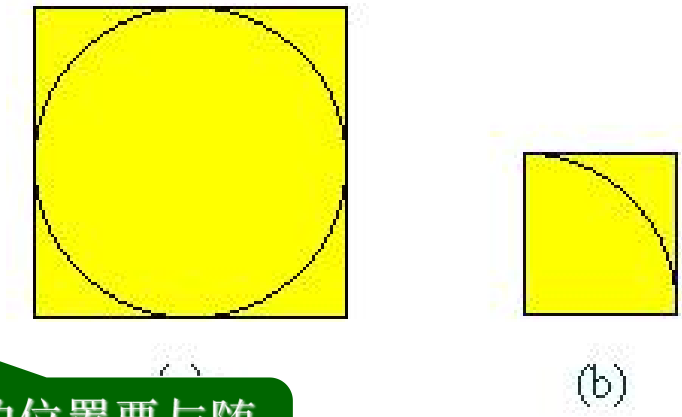
一定要有好的随机数

用随机投点法计算π值

设有一半径为r的圆及其外切四边形。向该正方形随机地投掷n个点。设落入圆内的点数为k。由于所投入的点在正方形上均匀分布，因而所投入的点落入圆内的概率为 $\frac{\pi r^2}{4r^2}=\frac{\pi}{4}$ 。所以当n足够大

时，k与n之比就逼近这一概率。从而 $\pi \approx \frac{4k}{n}$

```
double Darts(int n)
{ // 用随机投点法计算π值
    static RandomNumber dart;
    int k=0;
    for (int i=1;i <=n;i++) {
      double x=dart.fRandom();
      double y=dart.fRandom();
      if ((x*x+y*y)<=1) k++;
      }
    return 4*k/double(n);
}
```

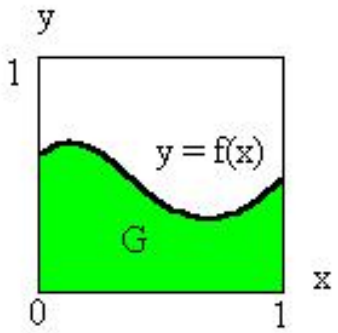


x,y的位置要与随机数的取值一致

计算定积分

设f(x)是[0，1]上的连续函数，且0≤f(x)≤1。

需要计算的积分为 $I=\int_0^1 f(x)dx$,积分I等于图中的面积G。



P239

在图所示单位正方形内均匀地作投点试验，则随机点落在曲线下面的概率为

$$P_r\{y \le f(x)\} = \int_0^1 \int_0^{f(x)} dydx = \int_0^1 f(x)dx$$

假设向单位正方形内随机地投入n个点(xi,yi)。如果有m个点落入G内，则随机点落入G内的概率 $I \approx \frac{m}{n}$

在游戏中，随机数的使用很普遍，注重提高局部的随机性

解非线性方程组（用随机方法求解）



求解下面的非线性方程组

$$\begin{cases} f_1(x_1, x_2, \cdots, x_n) = 0 \\ f_2(x_1, x_2, \cdots, x_n) = 0 \\ \cdots\cdots\cdots\cdots\cdots\cdots \\ f_n(x_1, x_2, \cdots, x_n) = 0 \end{cases}$$

其中，$x_1,x_2,\ldots,x_n$是实变量，f_i是未知量$x_1,x_2,\ldots,x_n$的非线性实函数。要求确定上述方程组在指定求根范围内的一组解 $x_1^*, x_2^*, \cdots, x_n^*$

在指定求根区域D内，选定一个随机点x_0作为随机搜索的出发点。在算法的搜索过程中，**假设第j步随机搜索得到的随机搜索点为x_j**。在第j+1步，计算出下一步的随机搜索**增量Δx_j**。从当前点x_j依Δx_j得到第j+1步的随机搜索点。当$x<\varepsilon$时，取为所求非线性方程组的近似解。否则进行下一步新的随机搜索过程。

P242程序分析

目标函数变化小时

有可能不能收敛

解非线性方程组



```
double fx = f(x,n);       //计算目标函数值
double min = fx;          //当前最优值

while(j<Steps)
{
    //(1)计算随机搜索步长
    if(fx<min)//搜索成功
    {
        min = fx;
        a *= k;
        success = true;
    }
    else//搜索失败
    {
        mm++;
        if(mm>M)
        {
            a /= k;
        }
        success = false;
    }
```

```
if(min<epsilon)
{
    break;
 }
```

两个方向，上一次有效果继续
上一次无效果终止？

```
//(2)计算随机搜索方向和增量
for(int i=1; i<=n; i++)
{
    r[i] = 2.0 * rnd.fRandom()-1;
}

if(success)
{
    for(int i=1; i<=n; i++)
    {
        dx[i] = a * r[i];
    }
}
else
{
    for(int i=1; i<=n; i++)
    {
        dx[i] = a * r[i] - dx[i];
    }
}
```

舍伍德(Sherwood)算法



设A是一个确定性算法，当它的输入实例为x时所需的计算时间记为$t_{A(x)}$。设X_n是算法A的输入规模为n的实例的全体，则当问题的输入规模为n时，算法A所需的平均时间为

$$\bar{t}_A(n) = \sum_{x \in X_n} t_A(x) / |X_n|$$

这显然不能排除存在x∈Xn使得 $t_A(x) >> \bar{t}_A(n)$ 的可能性。希望获得一个概率算法B，使得对问题的输入规模为n的每一个实例均有

$$t_B(x) = \bar{t}_A(n) + s(n)$$

这就是舍伍德算法设计的基本思想。当s(n)与$t_{A(n)}$相比可忽略时，舍伍德算法可获得很好的平均性能。

舍伍德(Sherwood)算法



复习学过的算法：（其随机算法为Sherwood）

（1）线性时间选择算法 P243

（2）快速排序算法

有时也会遇到这样的情况，即所给的算法无法直接改造成舍伍德型算法。此时可借助于随机预处理技术，不改变原有的算法，仅对其输入进行随机洗牌，同样可收到舍伍德算法的效果。

舍伍德(Sherwood)算法



```
Type select(Type a[], int lt, int rt, int k)
{
    // 计算a[lt:rt]中第k小元素
    static RandomNumber rnd;
    while(true)
    {
        if(lt > rt)
            return a[lt];
        int i = lt, j = lt+rnd.Random(rt-lt+1);    // 随机选择的划分基准
        Swap(a[i], a[j]);
        j = rt+1;
        Type pivot = a[lt];
        //以划分基准为轴作元素交换
        while(true)
        {
            while(a[++i] < pivot);
            while(a[--j] > pivot);
            if(i >= j)
                break;
            Swap(a[i], a[j]);
        }
```

用于打散数据，确定一个pivot

设一个基准
左边<基准
右边>基准

舍伍德(Sherwood)算法



```
    if(j - lt + 1 == k) return pivot;
    a[lt] = a[j];
    a[j] = pivot;
    // 对子数组重复划分过程
    if(j - lt + 1 < k)
        {
            k = k - j + lt - 1;
            lt = j + 1;
        }
    else
        rt = j - 1;
}
```

如果基准刚好是第k个

确定左边还是右边

舍伍德算法：如何合适的随机，才能达到较好的均衡性

舍伍德(Sherwood)算法:随机洗牌



用随机的方法，对确定性算法进行修改，目的是对任意的输入，所需要的求解时间相对平衡

例如，对于确定性选择算法，可以用下面的洗牌算法**shuffle**将数组a中元素随机排列，然后用确定性选择算法求解。这样做所收到的效果与舍伍德型算法的效果是一样的。

```
template<class Type>
void Shuffle(Type a[], int n)
{// 随机洗牌算法
  static RandomNumber rnd;
  for (int i=0;i<n;i++) {
    int j=rnd.Random(n-i)+i;
    Swap(a[i], a[j]);
    }
}
```

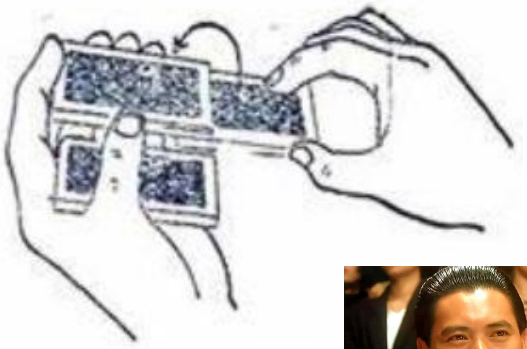
第i张都是与其后面的牌交换

还有没有其他方法？

舍伍德(Sherwood)算法:随机洗牌



如何模拟常见的洗牌算法?







搜索有序表



P245，P247代码分析

link[i]的值表示下一个元素的下标，从小到大

i	0	1	2	3	4	5	6	7
value[i]	极大值	2	3	13	1	5	21	8
link[i]	4	2	5	6	1	7	0	3

第一个数的位置



搜索有序表

如何用舍伍德算法，对排序的数组进行查找？

顺序：最坏情况在最后一个找到

增加随机：取几个随机数，找出最接近目标的那个，从这里开始

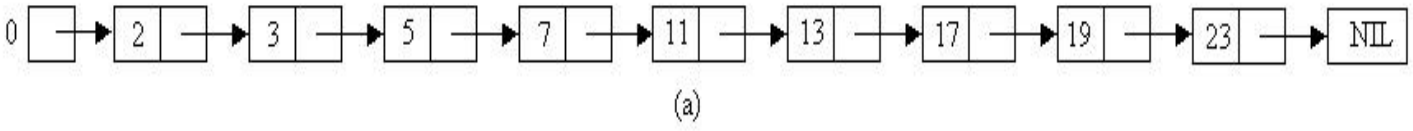


跳跃表

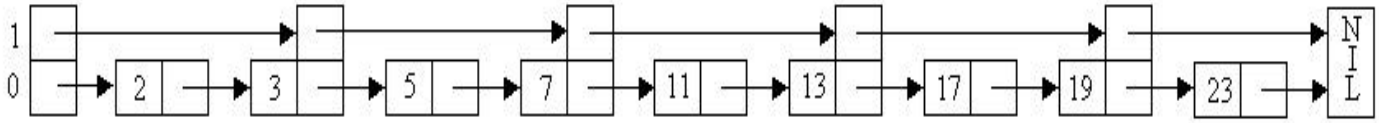
- 舍伍德型算法的设计思想还可用于设计高效的数据结构。
- 如果用有序链表来表示一个含有n个元素的有序集S，则在最坏情况下，搜索S中一个元素需要Ω(n)计算时间。
- 提高有序链表效率的一个技巧是在有序链表的部分结点处增设附加指针以提高其搜索性能。在增设附加指针的有序链表中搜索一个元素时，可借助于附加指针跳过链表中若干结点，加快搜索速度。这种增加了向前附加指针的有序链表称为跳跃表。
- 应在跳跃表的哪些结点增加附加指针以及在该结点处应增加多少指针完全采用随机化方法来确定。这使得跳跃表可在O(logn)平均时间内支持关于有序集的搜索、插入和删除等运算。

- 寻找某人-》学校-》学院-》班级-》确定

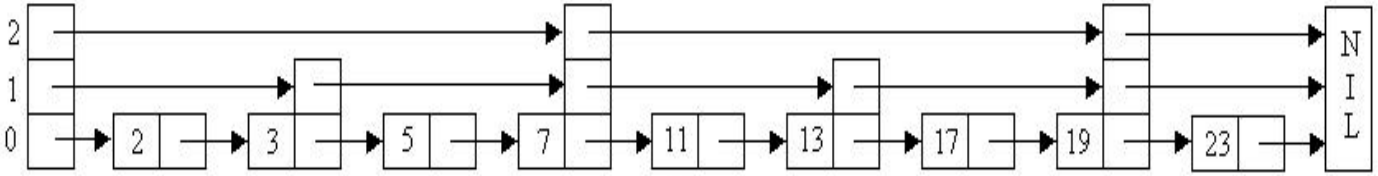
跳跃表


0
2
3
5
7
11
13
17
19
23
NIL

(a)


1
0
2
3
5
7
11
13
17
19
23
NIL

(b)


2
1
0
2
3
5
7
11
13
17
19
23
NIL

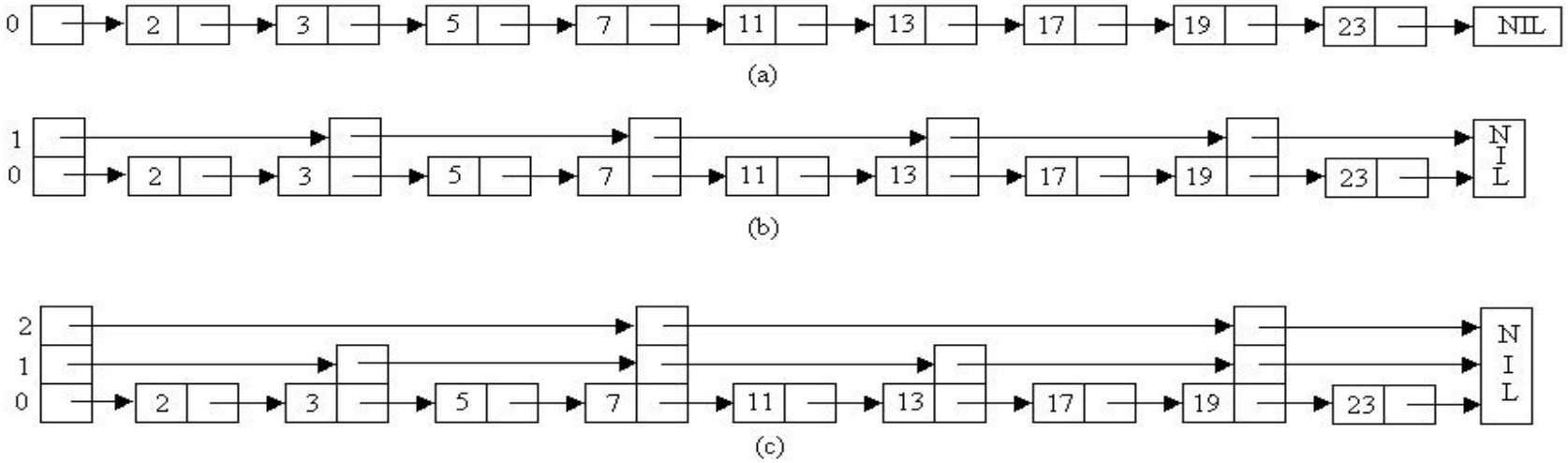
(c)



跳跃表

在一般情况下，给定一个含有n个元素的有序链表，可以将它改造成一个完全跳跃表，使得每一个k级结点含有k+1个指针，分别跳过2^k-1，$2^{k-1}-1$，…，2^0-1个中间结点。第i个k级结点安排在跳跃表的位置i2k处，$i \geq 0$。这样就可以在时间O(logn)内完成集合成员的搜索运算。在一个完全跳跃表中，最高级的结点是$\lceil \log n \rceil$级结点。

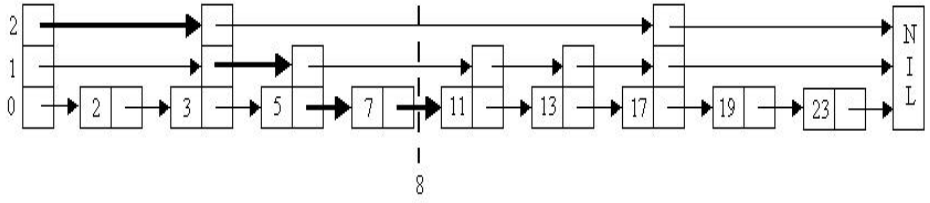
跳跃表

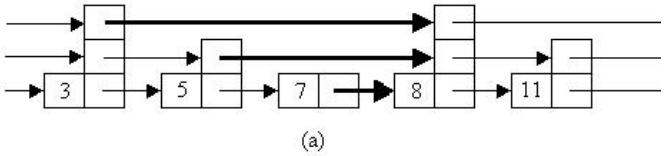


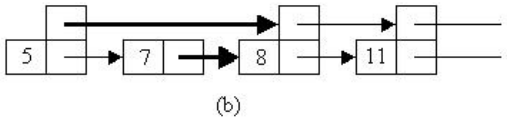
完全跳跃表与完全二叉搜索树的情形非常类似。它虽然可以有效地支持成员搜索运算，但**不适应于集合动态变化**的情况。集合元素的插入和删除运算会**破坏完全跳跃表原有的平衡状态**，影响后继元素搜索的效率。

跳跃表

为了在动态变化中维持跳跃表中附加指针的平衡性，必须使跳跃表中k级结点数维持在总结点数的一定比例范围内。注意到在一个完全跳跃表中，50%的指针是0级指针；25%的指针是1级指针；…；$(100/2^{k+1})$%的指针是k级指针。因此，在插入一个元素时，以概率1/2引入一个0级结点，以概率1/4引入一个1级结点，…，以概率$1/2^{k+1}$引入一个k级结点。另一方面，一个i级结点指向下一个同级或更高级的结点，它所跳过的结点数不再准确地维持在2^i-1。经过这样的修改，就可以在插入或删除一个元素时，通过对跳跃表的局部修改来维持其平衡性。







跳跃表



注意到，在一个完全跳跃表中，具有i级指针的结点中有一半同时具有i+1级指针。为了维持跳跃表的平衡性，可以**事先确定一个实数0<p<1**，并要求在跳跃表中维持在具有i级指针的结点中同时具有i+1级指针的结点所占比例约为p。为此目的，在插入一个新结点时，先将其结点级别初始化为0，然后用随机数生成器**反复地产生一个[0，1]间的随机实数q。如果q<p，则使新结点级别增加1，直至q≥p。**由此产生新结点级别的过程可知，所产生的新结点的级别为0的概率为*1-p(?)*，级别为1的概率为p(1-p)，…，**级别为i的概率为$p^i(1-p)$**。如此产生的新结点的**级别有可能是一个很大的数，甚至远远超过表中元素的个数**。为了避免这种情况，**用$\log_{1/p} n$作为新结点级别的上界**。其中n是当前跳跃表中结点个数。当前跳跃表中任一结点的级别不超过 $\log_{1/p} n$ 当p=0.5时，刚好 $\log n$

拉斯维加斯(Las Vegas)算法



拉斯维加斯算法的**一个显著特征**是它*所作的随机性决策有可能导致算法找不到所需的解。*

```
void obstinate(Object x, Object y)
  {// 反复调用拉斯维加斯算法LV(x,y)，直到找到问题的一个解y
    bool success= false;
    while (!success) success=lv(x,y);
  }
```

所以用一个循环找解

设p(x)是对输入x调用拉斯维加斯算法获得问题的一个解的概率。一个正确的拉斯维加斯算法应该对所有输入x均有p(x)>0。

设t(x)是算法**obstinate**找到具体实例x的一个解所需的平均时间，s(x)和e(x)分别是算法对于具体实例x求解成功或求解失败所需的平均时间，**则有**：

$$t(x) = p(x)s(x) + (1 - p(x))(e(x) + t(x))$$

解此方程可得：

$$t(x) = s(x) + \frac{1 - p(x)}{p(x)} e(x)$$



n后问题

对于n后问题的任何一个解而言，每一个皇后在棋盘上的位置无任何规律，不具有系统性，而更象是随机放置的。由此容易想到下面的拉斯维加斯算法。

在棋盘上相继的各行中随机地放置皇后，并注意使新放置的皇后与已放置的皇后互不攻击，直至n个皇后均已相容地放置好，或已没有下一个皇后的可放置位置时为止。

如果将上述随机放置策略与回溯法相结合，可能会获得更好的效果。可以先在棋盘的若干行中随机地放置皇后，然后在后继行中用回溯法继续放置，直至找到一个解或宣告失败。随机放置的皇后越多，后继回溯搜索所需的时间就越少，但失败的概率也就越大。

```
bool Queen::Place(int k)
{
    // 测试皇后k置于第x[k]列的合法性
    for(int j = 1; j < k; ++ j)
        if((abs(k-j) == abs(x[j]-x[k])) || (x[j]==x[k]))
            return false;
    return true;
}
```

可放置性判断

1)不同列：$x_i \neq x_j$
2)不处于同一正、反对角线：
$|i-j| \neq |x_i - x_j|$
P155，约束如何得到

```
while ((k<=n)&&(count>0)){
    for  (...)
      x[k]=i;
      if (Place(k)) y[count++]=i;
}
if (count>0) x[k++]=y[rnd.Random(count)];
```

?



反复调用拉斯维加斯算法，直至成功：
while（！X.QueensLV()）；
普通人下棋的时候，是不是有点拉斯维加斯？/ 悔棋

MIT 6.046J



改进有效性：
应用拉斯维加斯算法的思想改进快速排序，得到一种随机快速排序算法；
使得快速排序算法的时间复杂度稳定在O(nlogn)

https://zhuanlan.zhihu.com/p/81670606
https://ocw.mit.edu/courses/6-046j-design-and-analysis-of-algorithms-spring-2015/



蒙特卡罗(Monte Carlo)算法

蒙特卡洛模拟是在二战期间，当时在原子弹研制的项目中，为了模拟裂变物质的中子随机扩散现象，由美国数学家冯·诺伊曼（学计算机的同学都知道这位冯同志的大名，人称“计算机之父”）和乌拉姆等发明的一种统计方法。之所以起名叫蒙特卡洛模拟，是因为蒙特卡洛在是欧洲袖珍国家摩纳哥一个城市，这个城市在当时是非常著名的一个赌城。因为赌博的本质是算概率，而蒙特卡洛模拟正是以概率为基础的一种方法，所以用赌城的名字为这种方法命名。

- 蒙特卡洛模拟是在计算机上模拟项目实施了成千上万次，每次输入都随机选择输入值。由于每个输入很多时候本身就是一个估计区间，因此计算机模型会随机选取每个输入的该区间内的任意值，通过大量成千上万甚至百万次的模拟次数，最终得出一个累计概率分布图，这个就是蒙特卡洛模拟。

- 实际上，前面求面积的方法，是一个蒙特卡洛统计方法。
  - 1.近似计算不规则面积/体积/积分（面积）
  - 2.模拟随机过程，预测随机过程可能性结果的区间范围（赌博）
  - 3.结合接受-拒绝采样，对分布的未知参数进行统计推断



蒙特卡罗(Monte Carlo)算法

- 在实际应用中常会遇到一些问题，不论采用确定性算法或概率算法都无法保证每次都能得到正确的解答。蒙特卡罗算法则在一般情况下可以保证对问题的所有实例都以高概率给出正确解，但是通常无法判定一个具体解是否正确。
- 设p是一个实数，且1/2<p<1。如果一个蒙特卡罗算法对于问题的任一实例得到正确解的概率不小于p，则称该蒙特卡罗算法是p正确的，且称p-1/2是该算法的优势。
- 如果对于同一实例，蒙特卡罗算法不会给出2个不同的正确解答，则称该蒙特卡罗算法是一致的。
- 有些蒙特卡罗算法除了具有描述问题实例的输入参数外，还具有描述错误解可接受概率的参数。这类算法的计算时间复杂性通常由问题的实例规模以及错误解可接受概率的函数来描述。



蒙特卡罗(Monte Carlo)算法

对于一个一致的p正确蒙特卡罗算法，要提高获得正确解的概率，
只要执行该算法若干次，并选择出现频次最高的解即可。
如果重复调用一个一致的(1/2+ε)正确的蒙特卡罗算法2m-1次，得到正确解的概率至少为1-δ，其中，

$$\delta = \frac{1}{2} - \varepsilon \sum_{i=0}^{m-1} \binom{2i}{i} \left(\frac{1}{4} - \varepsilon^2\right)^i \le \frac{(1-4\varepsilon^2)^m}{4\varepsilon\sqrt{\pi m}}$$

对于一个解所给问题的蒙特卡罗算法MC(x)，如果存在问题实例的子集X使得：
(1)当$x \notin X$时，MC(x)返回的解是正确的；
(2)当$x \in X$时，正确解是y_0，但MC(x)返回的解未必是y_0。
称上述算法MC(x)是偏y_0的算法。

重复调用一个一致的，p正确偏y_0蒙特卡罗算法k次，可得到一个$(1-(1-p)^k)$正确的蒙特卡罗算法，且所得算法仍是一个一致的偏y_0蒙特卡罗算法。



主元素问题

设T[1:n]是一个含有n个元素的数组。当|{i|T[i]=x}|>n/2时，称元素x是数组T的主元素。

```
template<class Type>
bool Majority(Type *T, int n)
{// 判定主元素的蒙特卡罗算法
    int i=rnd.Random(n)+1;
    Type x=T[i];
    // 随机选择数组元素
    int k=0;
    for (int j=1;j<=n;j++)
        if (T[j]==x) k++;
    return (k>n/2);
    // k>n/2 时T含有主元素
}
```

对于任何给定的ε>0，算法**majorityMC**重复调用⌈log(1/ε)⌉次算法**majority**。它是一个偏真蒙特卡罗算法，且其错误概率小于ε。算法**majorityMC**所需的计算时间显然是O(nlog(1/ ε))。

```
template<class Type>
bool MajorityMC(Type *T, int n, double e)
{// 重复调用算法Majority
    int k=ceil(log(1/e)/log(2));
    for (int i=1;i<=k;i++)
      if (Majority(T,n)) return true;
    return false;
}   //P263改进
```

比较

比较项目	蒙特卡罗算法
规律	采样越多，越逼近最优解
例子	从不透明的苹果筐中挑最大的苹果
策略	尽量找大的，但不保证是最大的
适用情景（依问题而定）	要求在有限采样内，必须给出一个解，但不要求是最优解

比如：估算销量等（双11的各种估算/以前，为什么双11用各种方法让你放购物车）



拉斯维加斯算法

采样越多，越有可能找到最优解
从一串钥匙中试出能开锁的钥匙
尽量找最好的，但不保证能找到（除非全枚举）
对采样要求没有限制，要求必须给出最优解

一开始：拉斯维加斯
最后　：蒙特卡罗
最后一颗：拉斯维加斯和蒙特卡洛都获得了最优解，俗称【中奖】

蒙特卡罗



赌徒和庄家对赌抛硬币，如果为正面，本轮赌徒赢，庄家付给赌徒1元，结果为反面，本轮赌徒输，赌徒付给庄家1元。赌徒有初始赌本10元，手上的钱一旦输光则退出赌局，如何来模拟这个赌博过程?

我们首先来分析一下这个过程，赌徒的赌博结果本质上依托于每次抛掷硬币的结果，每一轮赌博就是一个伯努利试验，赢的概率是p=0.5，赌博的过程就是由这一串伯努利试验构成的伯努利随机过程，每轮赌局中，如果赢则赌本增加1元，输则赌本减少1元。

当然了，如果对某一个特定的赌徒，一旦开始进入赌局，则最终由每轮赌局结果构成的序列就是唯一的。那么如果我们想观察整个赌博过程的整体特征，我们该怎么办？好办，还是使用之前讲过的蒙特卡罗方法，采用大量的样本，最终观察样本结果的整体特征。

这里，我们为了说明问题，先采用的样本数也就是赌徒数为10个，轮数为100轮，也就是每个赌徒最多和庄家对赌100轮，如果在这个过程中输光了赌本，则提前退出，如果到100轮还有赌本，赌局也停止。

蒙特卡罗



person	round	money
1	64	0
2	100	14
3	90	0
4	100	16
5	88	0
6	100	14
7	100	6
8	78	0
9	52	0
10	18	0

在每轮赌局中，我们首先生成了一个随机变量，他在0和1当中等概论选取，模拟的就是抛掷硬币的过程，当结果为1时表示硬币为正面，本轮赢庄家1元，结果为0时表示硬币为反面，本轮输庄家1元。这里设定的是每个赌徒最多赌100轮，如果不到100轮就输光了，则退场，这样就模拟了整个赌博的随机过程。

从运行结果来看，10个赌徒中有6个提前输光退场，剩下的4个打满全场的人中，有3个是挣钱的，有1个是亏钱的。

蒙特卡罗



蒙特卡洛方法讲求大的样本量，我们把样本数和轮数都修改一下，并且统计一些指标。赌徒的总人数设置为1000000人，轮数设置为100/1000/10000，来观察样本总体的表现：

```
总轮数:100,总人数:100000
输光赌本提前出局的人数:31148
赌满全场且盈利的人数:44458
赌满全场且亏损的人数:23923
```

```
总轮数:1000,总人数:100000
输光赌本提前出局的人数:75154
赌满全场且盈利的人数:23441
赌满全场且亏损的人数:1386
```

```
总轮数:10000,总人数:100000
输光赌本提前出局的人数:91902
赌满全场且盈利的人数:8060
赌满全场且亏损的人数:38
```

从结果中不难发现，这种和庄家的对赌，随着轮数的增加，基本上都破产被收割了。即使庄家不出千，输赢概率各半，赌的越久，基本上都是输光破产走人。

为什么赌场提供酒店等服务？
为什么赌场借钱？

蒙特卡罗/采样



导航软件：

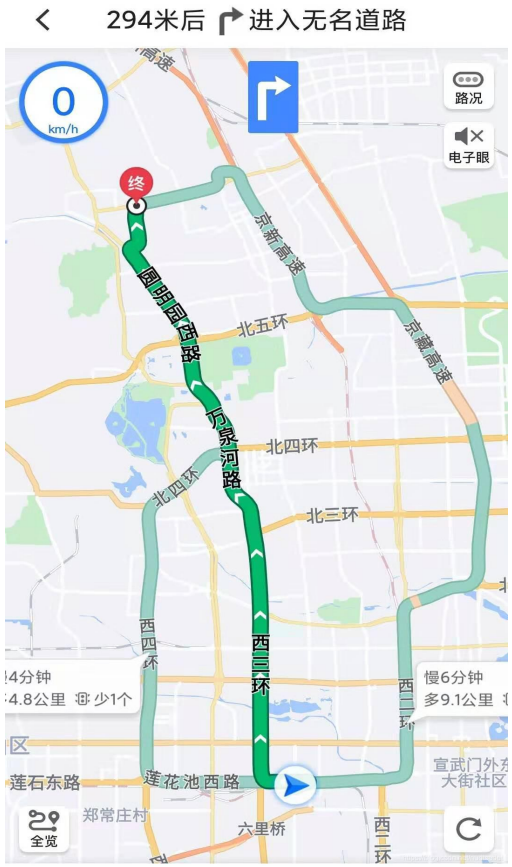
进入隧道无信号，如何导航？

如何估算到达目的地时间？

如何分析真假堵车（匝道赌）？

天气预报：

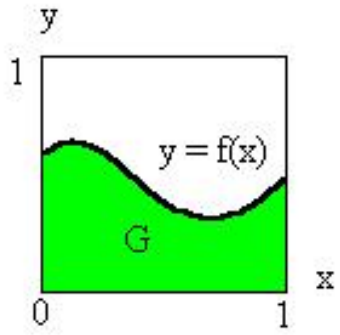


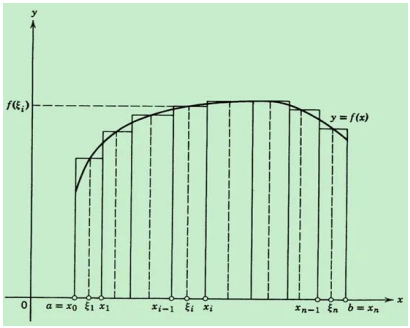


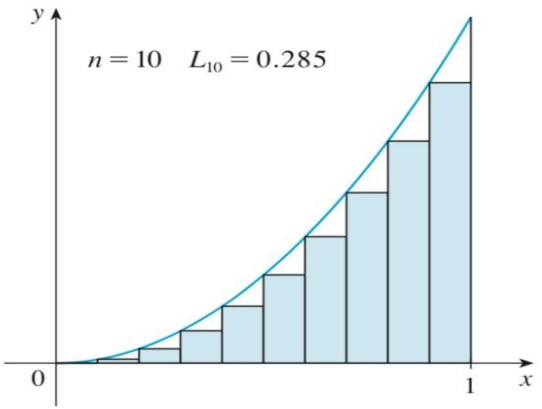


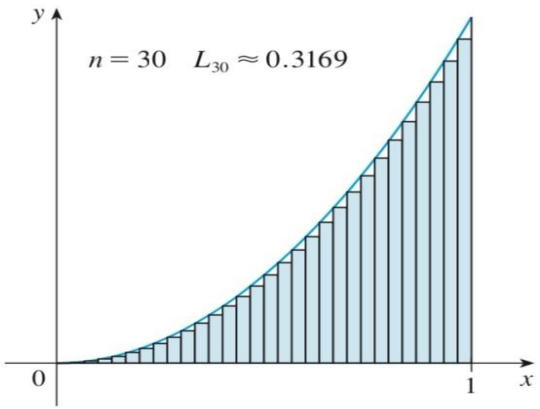
蒙特卡罗/采样

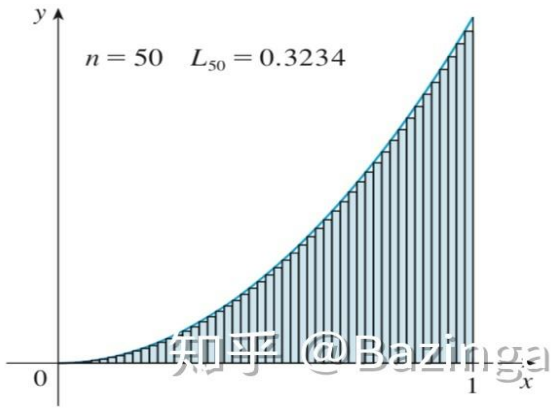
用微积分方法求解，也是M采样的方法







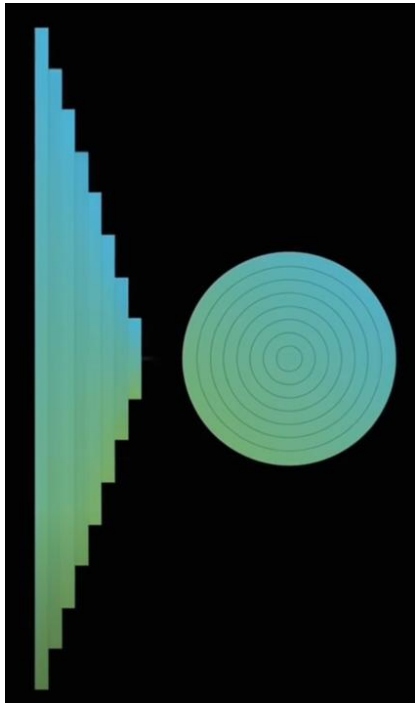


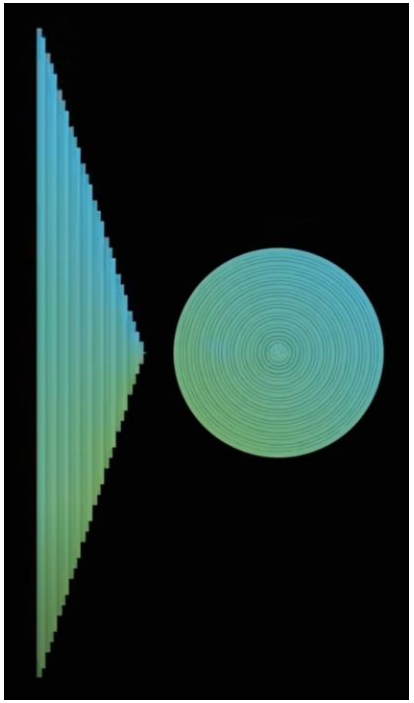


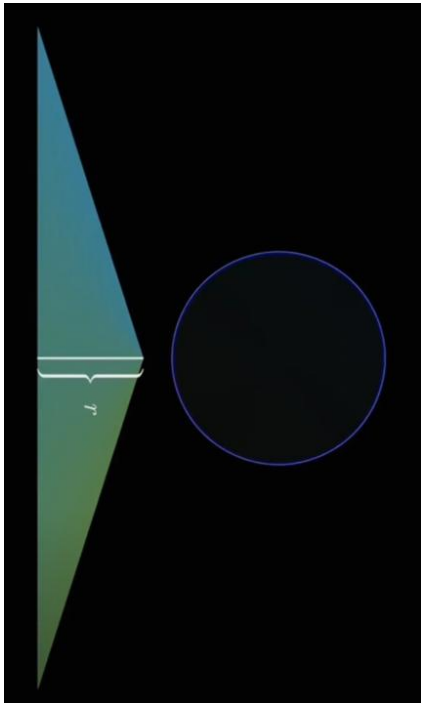
知乎 @Bazinga

蒙特卡罗/采样

圆的面积







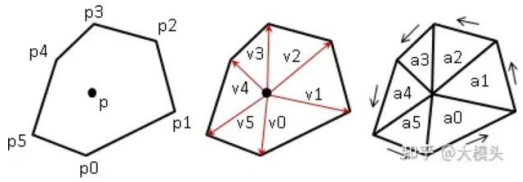
显微镜下的大明之丝绢案：妖田

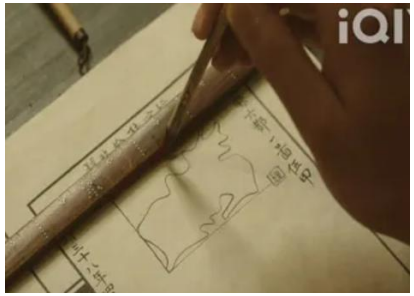




推步聚顶之术：

先牵经纬以衡量，再点圆初标步长，田型取顶分别数，再算推步知地方。





Example [edit]

For the area of the pentagon with

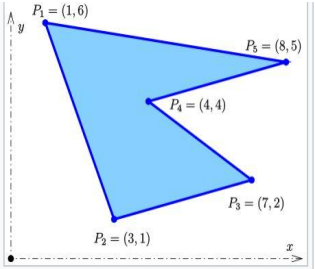
$$P_1 = (1,6), P_2 = (3,1), P_3 = (7,2),$$
$$P_4 = (4,4), P_5 = (8,5)$$

one gets

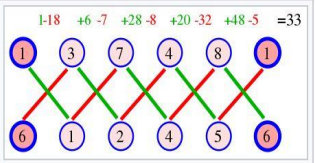
$$2A = \begin{vmatrix} 1 & 3 \\ 6 & 1 \end{vmatrix} + \begin{vmatrix} 3 & 7 \\ 1 & 2 \end{vmatrix} + \begin{vmatrix} 7 & 4 \\ 2 & 4 \end{vmatrix} + \begin{vmatrix} 4 & 8 \\ 4 & 5 \end{vmatrix} + \begin{vmatrix} 8 & 1 \\ 5 & 6 \end{vmatrix}$$
$$= 1 - 18 + 6 - 7 + 28 - 8 + 20 - 32 + 48 - 5 = 33$$
$$A = 16.5$$

The advantage of the shoelace form: Only 6 columns have to be written for calculating the 5 determinants with 10 columns.

Deriving the formulas [edit]



Example



Horizontal shoelace form for the example.

显微镜下的大明之丝绢案：妖田

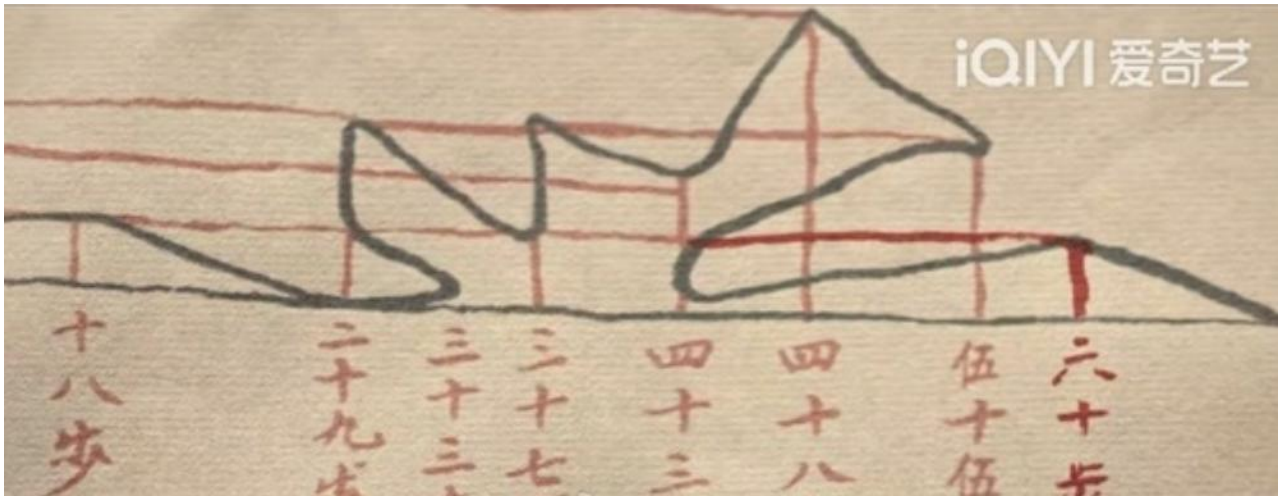




推步聚顶之术：

先牵经纬以衡量，再点圆初标步长，田型取顶分别数，再算推步知地方。

量纵横->标坐标->切割图形/分别算->算点间间隔/得到结果





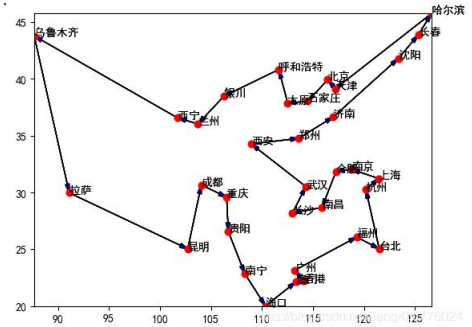


随机+遗传算法/AI/不确定

TSP/旅行商/中国卖货郎：

一个商品推销员要去若干个城市推销商品，该推销员从一个城市出发，
需要经过所有城市一次并且仅一次之后，回到出发城市。
问他应如何选择在城市之间的行程路线，以使他走过的总路程最短

对于一条成功的路径：信息素/权重



车辆路径问题(VRP,MDVRP,VRPTW)：

车辆路径问题（VRP）最早是由 Dantzig 和 Ramser 于1959年首次提出，
它是指一定数量的客户，各自有不同数量的货物需求，配送中心向客户
提供货物，由一个车队负责分送货物，组织适当的行车路线，目标是
使得客户的需求得到满足，并能在一定的约束下，达到诸如路程最短、
成本最小、耗费时间最少等目的。

